

LLM Workshop

Dr.-Ing. Grigory Devadze

2025-11-23



Dr.-Ing. Grigory Devadze

Consulting • Training • Softwareentwicklung • Data Science • ML • KI





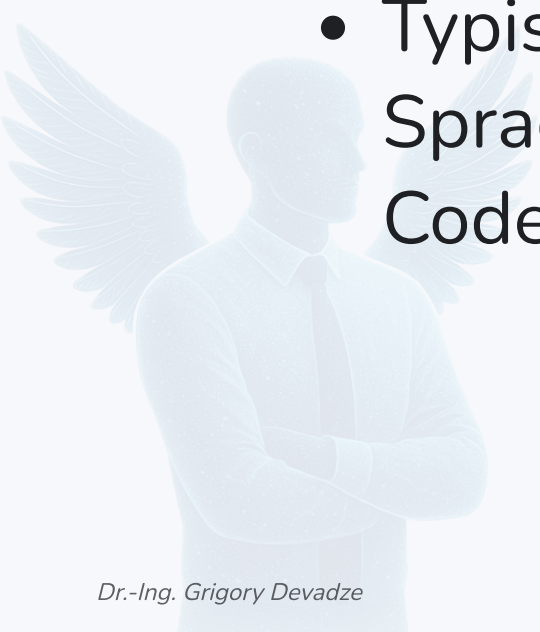
Was sind LLMs und wie funktionieren sie?



Was sind LLMs und wie funktionieren sie?

- LLM = Large Language Model, basierend auf Deep Learning, insbesondere der **Transformer-Architektur** (Vaswani et al., 2017)
- Kernmechanismus: **Self-Attention**
- Modelle lernen statistische Muster in Sprache und bilden **Wahrscheinlichkeitsverteilungen**
- Generieren Text durch Vorhersage des nächsten Tokens in einer Sequenz (autoregressive Generierung)

- Erfassen statistische Zusammenhänge, Bedeutungen und Strukturen in großen Textkorpora
- Werden auf Milliarden von Sätzen aus vielen Sprachen und Domänen vortrainiert
- Typische Aufgaben: Sprachverständnis, Sprachgenerierung, Übersetzung, Zusammenfassung, Codegenerierung, u.v.m.



Motivation: Warum LLMs?

- Ein Großteil des menschlichen Wissens liegt in Textform vor
- Heute gibt es riesige Mengen hochwertiger Textdaten, die genutzt werden können
- Immer mehr neue Textdaten: Webseiten, wissenschaftliche Literatur, Fachartikel



- Textdaten enthalten wiederkehrende Muster und semantische Strukturen, die sich sehr gut für maschinelles Lernen eignen.
- Ziel: Automatisierung, Assistenzsysteme und Analysewerkzeuge für komplexe Textdaten.



Motivation: Warum LLMs? (Fortsetzung)

- Anwendungen: Chatbots, Übersetzer, Zusammenfassungen, Text-Analysertools
- Einfacheres (Vor-)Filtern, Analysieren und Strukturieren riesiger Informationsmengen
- Automatisierung von Wissensarbeit: Contenterstellung, Kundenservice, medizinische Dokumentation, Codegenerierung
- Wettbewerbsdruck: KI-basierte Lösungen verschieben Marktstandards und Nutzererwartungen schnell
- Unternehmen müssen mitziehen, um effizient zu bleiben

Grundidee moderner KI-Modelle (Transformer)

- Moderne KI-Modelle können viele Wörter eines Textes **gleichzeitig** betrachten
- Dadurch verstehen sie **Zusammenhänge im ganzen Satz** besser
- Sie erkennen, **welche Wörter wichtig** sind
- Diese Technik ermöglicht es KI, **Texte zu verstehen, zu schreiben und zusammenzufassen**

Tokenisierung

- Text wird in **Tokens** zerlegt, damit ein Modell ihn verarbeiten kann
- Tokens können **Wörter, Wortteile oder Buchstaben** sein



- Beispiel: “*unbreakable*” → [“un”, “break”, “able”]
- Häufige Wörter bleiben ganz, seltene werden in **Wortstücke** zerlegt
- Die Tokenisierung beeinflusst **Effizienz, Kosten und verfügbare Kontextlänge**
- Vokabulargrößen reichen je nach Modell von **ca. 32k bis 200k Tokens**

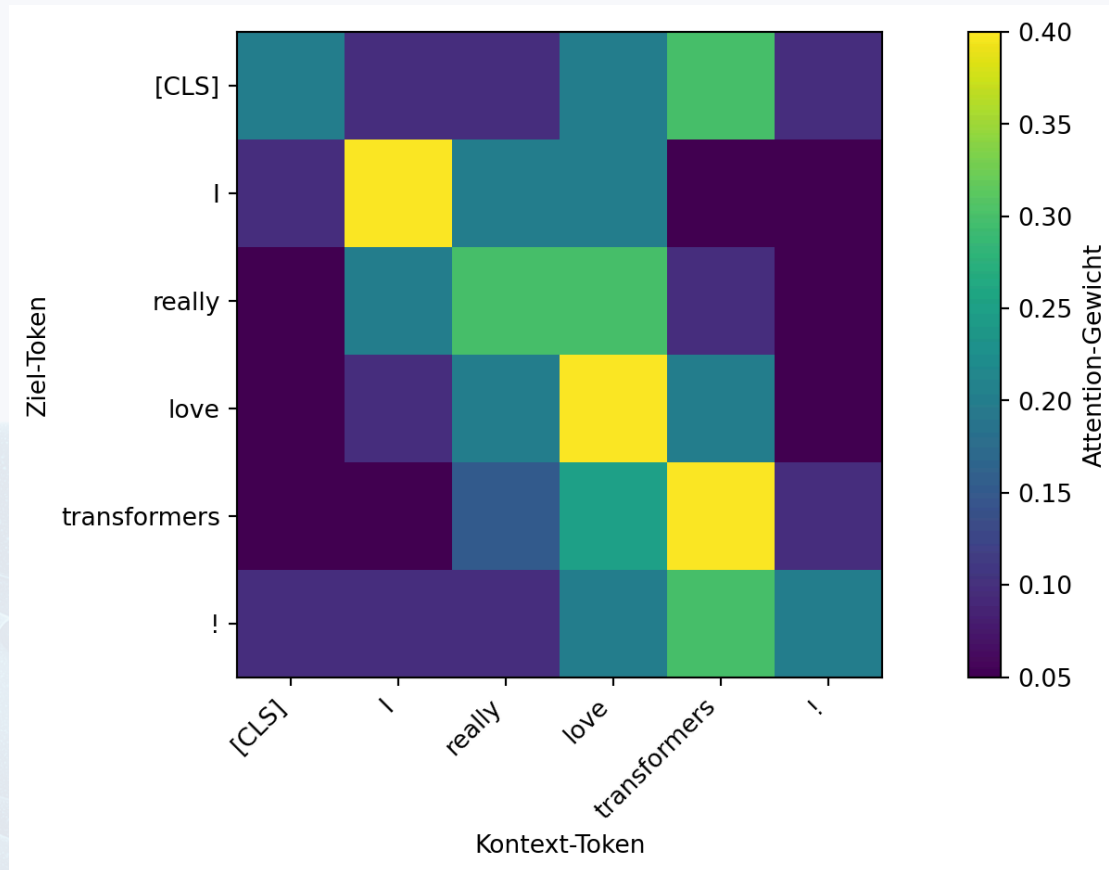
Self-Attention Mechanismus

- Jedes Token wird in drei Repräsentationen umgewandelt: **Query (Q)**, **Key (K)** und **Value (V)**
- Q und K bestimmen, **wie relevant** andere Tokens für dieses Token sind
- Das Relevanzmaß ergibt sich aus dem **Dot-Product**:
$$Q \cdot K^T$$

- Der Softmax normalisiert diese Scores zu **Gewichten**, die zusammen 1 ergeben
- Die Ausgabe ist eine **gewichtete Summe** der Value-Vektoren
- Formel: $Attention(Q, K, V) = softmax(QK^T / \sqrt{d}) \cdot V$
- d Dimension der Query- und Key-Vektoren
- **Multi-Head Attention** berechnet mehrere solcher Beziehungen parallel

Self-Attention – Visualisierung

Beispiel: Wie stark achtet jedes Wort auf die anderen Wörter im Satz?



Autoregressive Generierung

- LLMs erzeugen Text **schrittweise – Token für Token**
- Bei jedem Schritt berechnet das Modell: Wie **wahrscheinlich** ist jedes mögliche nächste Token?
- Mathematisch: $P(token_t | token_1, \dots, token_{t-1})$
- Die Modell-Ausgabe sind **Logits** (Rohscores), noch keine Wahrscheinlichkeiten

- Die **Softmax-Funktion** wandelt diese Logits in Wahrscheinlichkeiten um
- Ein Decoding-Verfahren wählt das nächste Token aus
- Die **Temperatur** beeinflusst die Kreativität: niedrig → stabil/deterministisch, hoch → variabler/„kreativer“



Kontext-Fenster und Skalierung

- Das **Kontext-Fenster** bestimmt, wie viele Tokens ein Modell gleichzeitig berücksichtigen kann
- Moderne Modelle erreichen **128k bis über 1 Million Tokens**
- Klassische Self-Attention hat eine **$O(n^2)$ -Komplexität** in der Sequenzlänge n
- Das bedeutet: Doppelte Kontextlänge → vierfache Rechenkosten

- Dies begrenzt in der Praxis die maximale Kontextgröße und die Inferenzgeschwindigkeit
- **FlashAttention** optimiert die Berechnung und reduziert Speicherverbrauch
- **Sparse- und effiziente Attention-Varianten** reduzieren die Anzahl der Interaktionen zwischen Tokens und ermöglichen größere Kontexte



Embedding Space

- Tokens werden in **hohe-dimensionalen Vektoren** abgebildet (typisch 768–12 288 Dimensionen)
- Diese Vektoren stellen die **Bedeutung** eines Tokens in Zahlenform dar
- Moderne LLMs erzeugen **kontextabhängige Embeddings** (z.B. „Bank“ $\neq$ „Bank“ je nach Satz)

- Ähnliche Bedeutungen liegen im Vektorraum **nahe beieinander**
- Klassisches Intuitionsbeispiel: „König“ – „Mann“ + „Frau“ $\approx$ „Königin“ (word2vec)
- Transformer-Schichten verändern diese Vektoren **schrittweise** weiter
- Jede Schicht extrahiert neue **Merkmale und Beziehungen** aus dem Kontext

Emergente Fähigkeiten

- Große Sprachmodelle entwickeln ab einer bestimmten Größe **neue Fähigkeiten**, die vorher kaum vorhanden waren
- **In-Context Learning:** Lernen aus Beispielen direkt im Prompt, ohne Gewichtsupdate
- **Chain-of-Thought:** Schrittweise Erklärung und Problemlösung

- **Instruction Following:** Befolgen komplexer Anweisungen durch Instruction-Tuning
- Diese Fähigkeiten werden mit zunehmender Modellgröße **stabiler und zuverlässiger**
- Skalierung, hochwertige Daten und Instruction-Tuning verstärken diese Effekte
- Ergebnis: Modelle können Aufgaben lösen, die nie explizit trainiert wurden

Limitierungen

- LLMs besitzen **kein echtes Verständnis** oder Kausalität – sie modellieren statistische Muster
- Sie können **plausibel klingende, aber falsche Inhalte** erzeugen (Halluzinationen)
- Es gibt **keine eingebaute Faktenprüfung** oder Verifikation der eigenen Aussagen
- Ein **Recency Bias** führt dazu, dass neuere Tokens im Kontext stärker gewichtet werden

- Modelle können ihr Wissen nach dem Training **nicht selbst aktualisieren**
- Faktenänderungen erfordern ein erneutes **Training oder Fine-Tuning**
- Die Modellantworten hängen stark von **Prompting, Kontext und Sampling-Parametern** ab





Training von LLMs

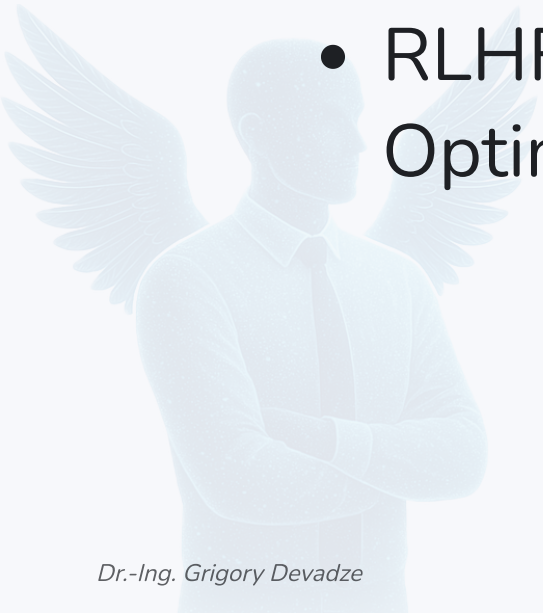


Training von LLMs

- Selbstüberwachtes Lernen: Vorhersage des nächsten Tokens (Next Token Prediction)
- Trainingskorpus: Milliarden bis Billionen Tokens aus Webseiten, Büchern, wissenschaftlichen Artikeln, Code, Foren etc.



- Pre-Training: Modell lernt allgemeine Sprachmuster und Weltwissen
- Supervised Fine-Tuning (SFT): Anpassung an spezifische Aufgaben durch Beispiele
- RLHF (Reinforcement Learning from Human Feedback): Optimierung durch menschliches Feedback



Parameter und praktische Auswirkungen

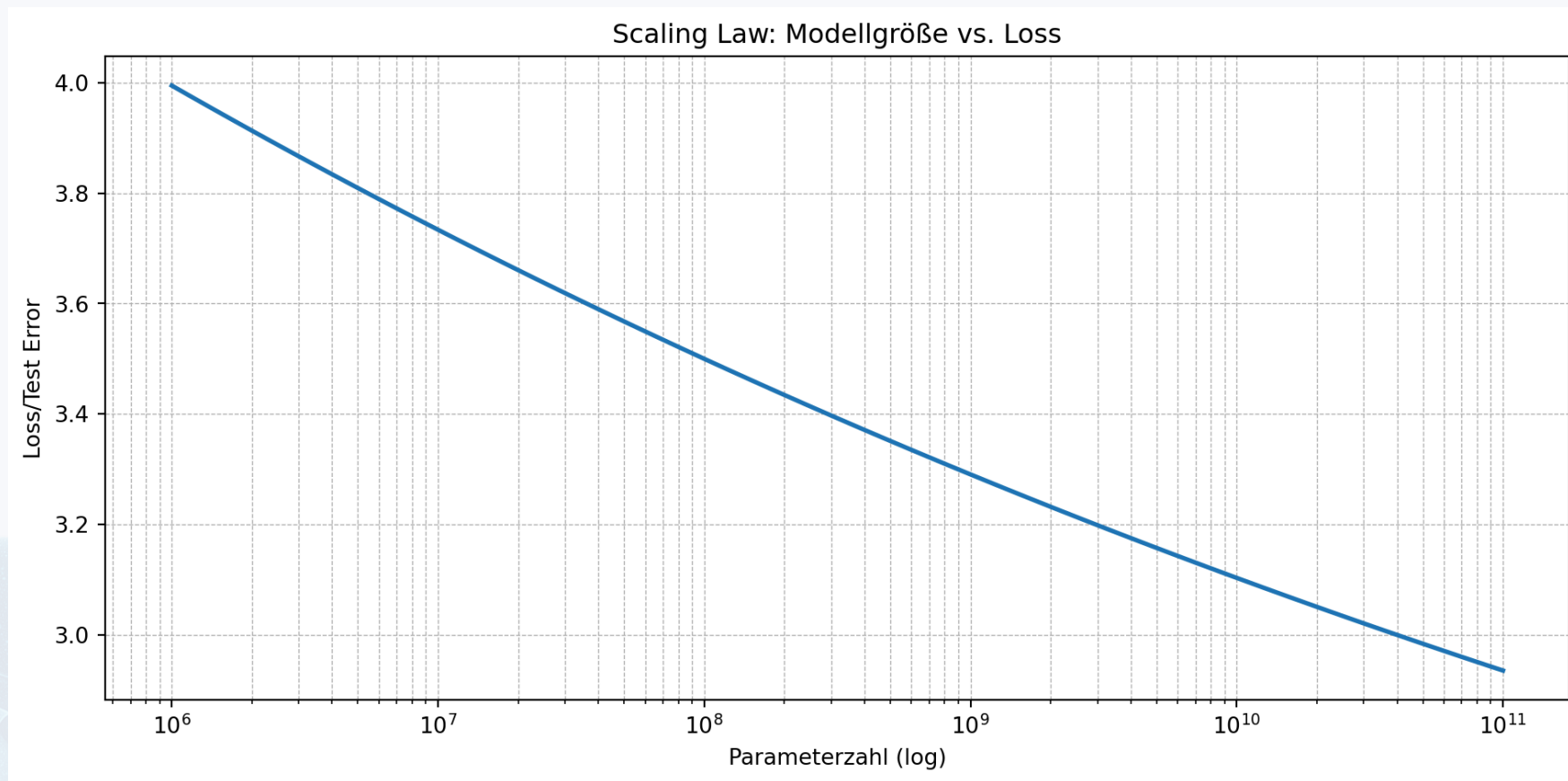
- Parameter = “Gedächtnis” des Modells (abgespeicherte statistische Bedeutungen)
- Kleine Modelle: 1–10 Mrd. Parameter (Llama 3 8B, Mistral 7B)
- Mittlere Modelle: 10–100 Mrd. (Llama 3 70B, Mixtral 8x7B)
- Große Modelle: 100+ Mrd. (GPT-4/4.1/5, Claude 3 Opus)

- Mehr Parameter $\approx$ bessere Fähigkeiten (aber nicht linear)
- Größere Modelle: mehr Weltwissen, besseres Reasoning, komplexere Aufgaben
- Modellgröße, Datenmenge und Rechenleistung bestimmen die Leistungsfähigkeit (Scaling Laws, Kaplan et al. 2020)
- Emergente Fähigkeiten erscheinen erst ab kritischer Modellgröße



Scaling Laws für LLMs (Kaplan et al. 2020)

- **Skalierungsgesetze** zeigen: „Mehr Daten“ und „größere Modelle“ bringen fast immer bessere Ergebnisse (geringere Loss/Fehler), aber mit abnehmendem Grenznutzen.
- Das ursprüngliche Ergebnis: Die Test-Fehlerkurve fällt ungefähr im Potenzgesetz, solange Rechenaufwand, Daten und Modellgröße zusammen wachsen.



Hardware- und Infrastruktur-Anforderungen

- VRAM-Bedarf (Beispiele, FP16/4-bit):

Modellgröße	Präzision	VRAM benötigt
7B	FP16	~14 GB
7B	4-bit	~4 GB
70B	FP16	~140 GB
70B	4-bit	~38 GB

- Inferenz: Kleine Modelle (7B) auf Consumer-GPU (z.B. RTX 4090, 24 GB VRAM) möglich
- Training: Erfordert Cluster mit High-End-GPUs (A100, H100)

- Inferenz-Runtimes wie **vLLM**, **llama.cpp** oder **TGI** verbessern Geschwindigkeit und Speicherbedarf
- Software-Stack: **PyTorch**, **Transformers**, **vLLM**, **GGUF/GGML**, **llama.cpp**, **LLaMA Factory**





LLMs mit eigenen Daten verwenden



1. Retrieval-Augmented Generation (RAG)

- Externe Wissensdatenbank (z.B. Vektordatenbank) wird mit eigenen Dokumenten befüllt
- Dokumente werden vektorisiert (Embeddings) und bei Anfrage relevante Passagen abgerufen
- LLM erhält Kontext aus der Datenbank und generiert darauf basierend die Antwort
- Vorteile: Keine Modelländerung nötig, aktuelle und domänenspezifische Informationen möglich

2. Fine-Tuning

- Vortrainiertes Modell wird gezielt auf eigene Daten/Anwendungsfälle weitertrainiert
- Parameter-Efficient Fine-Tuning (PEFT)
- Benötigt sorgfältige Datenaufbereitung und Qualitätskontrolle
- Relativ wenig Daten und kurze Trainingszeit reichen oft für große Effekte
- Ergebnis: Modell liefert bessere Resultate für die gewünschte Aufgabe

3. Prompt Engineering

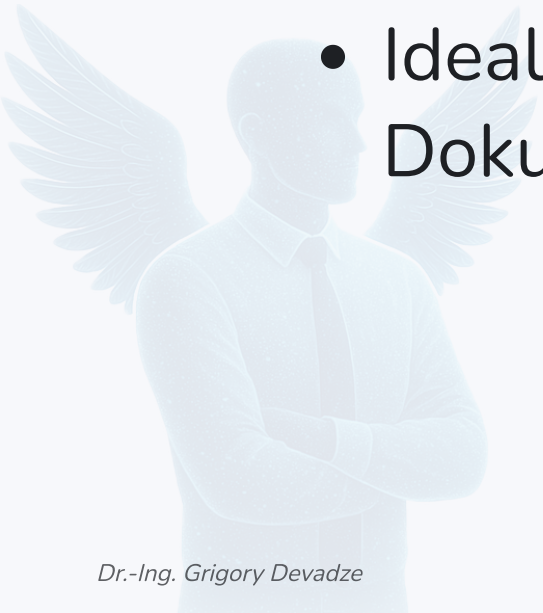
- Optimierte Prompts, Few-Shot Learning, In-Context Learning, Chain-of-Thought Prompting
- Keine Modelländerung, sondern gezielte Steuerung durch den Input
- Effektiv für viele Aufgaben, insbesondere bei leistungsfähigen Modellen



4. Map-Reduce / Hierarchical Summarization

- Große Dokumente werden in **kleine Chunks** aufgeteilt
- Das LLM erstellt zunächst **Teilzusammenfassungen** („Map“-Phase)
- Diese werden anschließend zu einer **Gesamtzusammenfassung** verdichtet („Reduce“-Phase)

- Vorteil: Kann **lange Texte** verarbeiten, die nicht in das Kontextfenster passen
- Liefert strukturierte, konsistente Zusammenfassungen großer Dokumente
- Ideal für: Reports, Verträge, Studien, technische Dokumentation, Forschungsberichte



Model Distillation

- Ziel: Ein großes, leistungsfähiges Modell („Teacher“) trainiert ein kleineres, schnelleres Modell („Student“).
- Der Student lernt nicht nur von „richtigen Tokens“, sondern direkt von den Wahrscheinlichkeiten und Ausgaben des Teachers.
- Dadurch kann der Student oft einen Großteil der Teacher-Leistung bei viel geringerer Größe/Komplexität erreichen.

Ablauf:

1. Trainiere ein großes Teacher-Modell (z.B. ein vollständiges LLM).
2. Erzeuge „Pseudo-Ausgabe“ durch Teacher-Inferenz.
3. Trainiere das kleine Student-Modell, indem es den Output des Teachers imitiert.



Vorteile:

- Kleine, schnelle Modelle werden möglich—perfekt für Edge, Mobilgeräte und Echtzeit-Anwendungen.
- Weniger Speicher, geringerer Energieverbrauch, kürzere Antwortzeiten.
- Distillation ist Standard bei LLM-Deployment („TinyLlama“, „TinyGPT“ etc.).



Kommerzielle Modelle im Vergleich

- **OpenAI – GPT-5**

- Flaggschiffmodell, vorgestellt am 7. August 2025.
- Multimodal, stark in Reasoning, Code und Sprache; nutzt adaptives Modell-Routing („fast“ vs „thinking“)
- Verfügbar für Plus/Pro/Team und über API.

- **DeepSeek – DeepSeek V3 & R1**

- V3: Open-Source-Alternative mit hoher Performance in Text & Code
- R1: Multimodal, effizient und anwendungsorientiert

- **Google – Gemini**
 - Gemini Ultra: Große Kontextfenster, starke Multimodalität, nahtlose Integration ins Google-Ökosystem
- **Anthropic – Claude 3 Familie (Haiku, Sonnet, Opus)**
 - Opus: Sehr großes Kontext-Fenster (200 k+ Tokens), hohes Textverständnis und Reasoning
- **Cohere – Command / Command R**
 - Enterprise-Fokus: Datenschutz, Kontrolle, Zuverlässigkeit

Open Source Modelle und deren Einsetzbarkeit

- **Llama 3 (Meta) – Varianten: 8 B & 70 B Parameter**
 - Offen veröffentlichte Modelle (8 B & 70 B) im April 2024.
 - Eignet sich für Text, Code und Open-Use-Szenarien.
- **8 B Variante:** Ideal für On-Premise und Entwickler-Setups mit geringerer Hardware.
- **70 B Variante:** Für anspruchsvollere Aufgaben und größere Kontexte.

- Beide Modelle nutzen laut Meta über **15 Billionen Tokens** beim Training.
- Lizenz-Hinweis: Zwar offen verfügbar, aber mit Nutzungsbedingungen – „open source“ nicht vollständig frei im OSI-Sinne. :
- Einsetzbarkeit: Lokale Deployment-Optionen vorhanden, aber Hardwareanforderungen steigen mit Größe.
- Empfehlung: 8 B für Exploration/Prototyping, 70 B wenn höhere Leistung benötigt wird.

Open Source Modelle und deren Einsetzbarkeit (Fortsetzung)

- Mistral 7B & Mixtral 8×7B (Mistral AI)
 - Mixtral 8×7B ist ein Sparse Mixture-of-Experts (SMoE) Modell mit ~46.7 B Parametern, aktiv ~12.9 B pro Token.
 - Lizenziert unter Apache 2.0 – echte Open Source Einsatzmöglichkeit.
- Mistral 7B: kleineres Modell mit geringeren Hardwareanforderungen.

- Mixtral verspricht laut Angaben „stärkere Ergebnisse bei vielen Benchmarks“ gegenüber älteren Open-Source-Modellen.
- Einsetzbarkeit: 7B Varianten für lokale Effekte, Mixtral-Modell für Leistung bei besseren Ressourcen.
- Lizenz- und Community-Vorteil: Apache 2.0 bietet hohe Flexibilität für eigene Projekte.



Tests mit Prompts

- Modelle werden mit **identischen, standardisierten Prompts** verglichen
- Abdeckung verschiedener Aufgabenklassen: Kreativität, Faktenwissen, Logik, Mathematik, Code, Zusammenfassungen
- Faire Vergleiche erfordern gleiche Einstellungen (z.B. Temperatur, Sampling, Kontext)

- Beispiel-Prompt (Kreativität): „Schreibe eine Kurzgeschichte über eine KI in der Zukunft.“
- Beispiel-Prompt (Erklärung): „Erkläre den Unterschied zwischen Quanten- und klassischen Computern.“
- Beispiel-Prompt (Mathe/Reasoning): „Löse die folgende Aufgabe: ...“
- Beispiel-Prompt (Code/Summary): „Generiere Python-Code für ...“ / „Fasse folgenden Text zusammen: ...“

Reaktion der Modelle hinsichtlich Kreativität, Relevanz und Verständnis des Kontexts

- Kreativität: Wie originell und abwechslungsreich sind die Antworten?
- Relevanz: Wie gut passt die Antwort zur gestellten Frage?
- Kontextverständnis: Kann das Modell Informationen aus dem Prompt korrekt aufnehmen und weiterverarbeiten?

- Faktentreue: Sind die Antworten korrekt und nachvollziehbar?
- Sprachfluss: Wie natürlich und flüssig ist die Sprache?



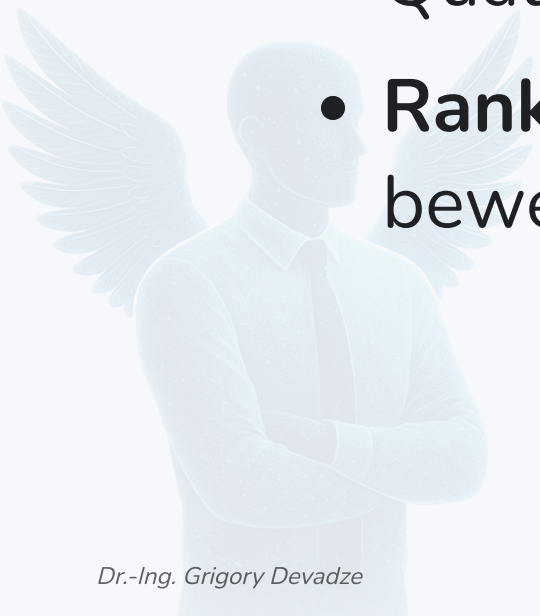
Bias testen: Kontroverse Themen gezielt behandeln

- Prompts zu politischen, gesellschaftlichen oder ethischen Themen (z.B. Gender, Religion, Politik)
- Beobachtung: Wie neutral, ausgewogen oder voreingenommen sind die Antworten?
- Unterschied beachten: **Opinion Bias** (Haltung) vs. **Fact Bias** (fehlerhafte oder verzerrte Fakten)
- Test: Werden bestimmte Meinungen bevorzugt? Gibt es Tendenzen zu bestimmten Sichtweisen?
- Dokumentation von auffälligen Biases und Halluzinationen

Bewertungsbogen mit Kriterien

- **Kohärenz:** Ist die Antwort logisch aufgebaut und in sich schlüssig?
- **Faktentreue:** Stimmen die Aussagen mit verlässlichen Informationen überein?
- **Sprachfluss:** Wie natürlich, flüssig und klar ist die Formulierung?
- **Bias:** Zeigt das Modell erkennbare Verzerrungen?
- **Kreativität:** Wie originell und abwechslungsreich ist die Antwort?

- **Kontextverständnis:** Wie gut verarbeitet das Modell die Informationen aus dem Prompt?
- **Konfidenz:** Passt das Auftreten der Antwort (Selbstsicherheit) zur tatsächlichen inhaltlichen Qualität?
- **Ranking:** Modelle werden pro Kriterium (z.B. 1–5) bewertet und anschließend verglichen



Model Evaluierung

- Evaluierung:
 - Benchmarks:
 - MMLU (Massive Multitask Language Understanding)
 - IFEval (Instruktionsbefolgung)
 - Menschliche Bewertung: Qualität, Faktentreue, Bias, Kreativität
 - Automatisierte Metriken: Perplexity (Güte der Wahrscheinlichkeitsverteilung), Accuracy (Multiple-Choice- oder Klassifikationsaufgaben)

Komplett eigene LLMs trainieren



Ablauf

1. Datensammlung und -aufbereitung (Webscraping, Bücher, wissenschaftliche Artikel, Code, Foren, etc.)
2. Tokenisierung und Vokabularerstellung
3. Architektur-Design (Transformer, Anzahl Layer, Heads, Embedding-Größe, Parameterzahl)
4. Pre-Training auf riesigen Textmengen (Self-Supervised Learning)
5. Optional: Supervised Fine-Tuning auf spezifischen Aufgaben/Domänen

6. Optional: RLHF (Reinforcement Learning from Human Feedback) für Feinschliff
7. Evaluation und Benchmarking
8. Deployment (Bereitstellung als API, On-Premise, Cloud, Edge)



Databricks Mosaic AI

- Kommerzielle und Open-Source-Plattform für effizientes LLM-Training
- Features: Distributed Training, Checkpointing, Data Streaming, Optimierungen für Kosten und Geschwindigkeit
- Unterstützt Training von Modellen mit Hunderten Milliarden Parametern

Hugging Face Parquet Daten (Training)

- Parquet: Spaltenbasiertes, komprimiertes Datenformat, geeignet für große Textmengen
- Hugging Face Datasets: Viele offene Datensätze im Parquet-Format, einfaches Laden und Sharding
- Vorteile: Weniger RAM, hohe I/O-Effizienz, einfache Integration in Trainingspipelines

Checkliste: Worauf achten beim Einsatz von LLMs im Unternehmen?

- **Daten & Datenschutz**
 - Welche Daten dürfen in die Modelle? (DSGVO, Betriebsgeheimnisse, personenbezogene Daten)
 - Werden Daten an externe Anbieter gesendet?
- **Security & Zugriffsschutz**
 - Zugriffskontrolle (z.B. API-Keys)
 - Schutz vor Prompt Injection, Leakage von sensiblen Informationen

- **Kosten & Skalierung**

- Inferenzkosten vs. Nutzung (Tokens, API-Kosten, GPU-Bedarf)
- Skalierungsstrategie: Cloud, On-Premise, Hybrid, Edge

- **Modellwahl & Einsatzszenario**

- Offene Modelle (On-Prem) vs. kommerzielle Modelle (API)
- Benötigte Fähigkeiten: Kontextlänge, Code, Multimodalität, Domänenverständnis

- **Qualität & Evaluierung**

- Testen auf Faktentreue, Bias, Stabilität, Robustheit
- Benchmarks + eigene Tests + menschliche Bewertung



- **Governance & Compliance**

- Dokumentation der Prompts, Datenquellen, Versionen und Modellentscheidungen
- Richtlinien für Einsatz, Monitoring und Freigaben

- **Monitoring & Betrieb**

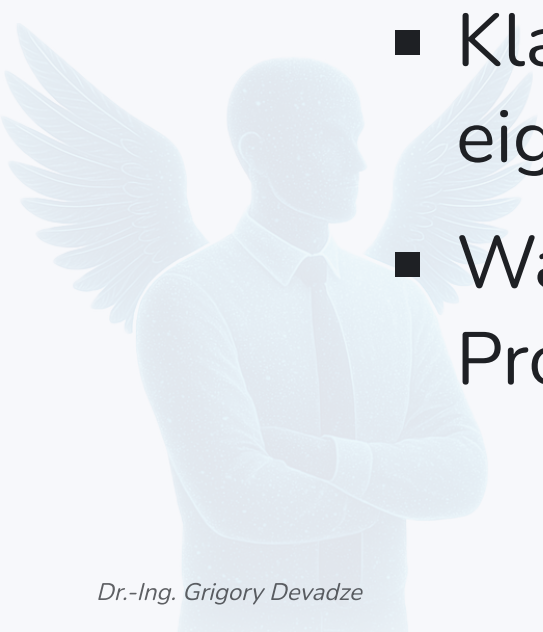
- Logging der Eingaben/Ausgaben (anonymisiert)
- Drift-Erkennung: Ändert sich das Verhalten des Modells?
- Incident-Handling bei Fehlverhalten des Modells

- **Risiken & Limitierungen**

- Umgang mit Halluzinationen
- Kein vollautomatisches Vertrauen: „Human-in-the-Loop“

- **Weiterentwicklung**

- Klare Strategie: Prompting, RAG, Fine-Tuning oder eigene Modelle?
- Wartung von Datenpipelines, Embeddings, Promptbibliotheken





Anhang

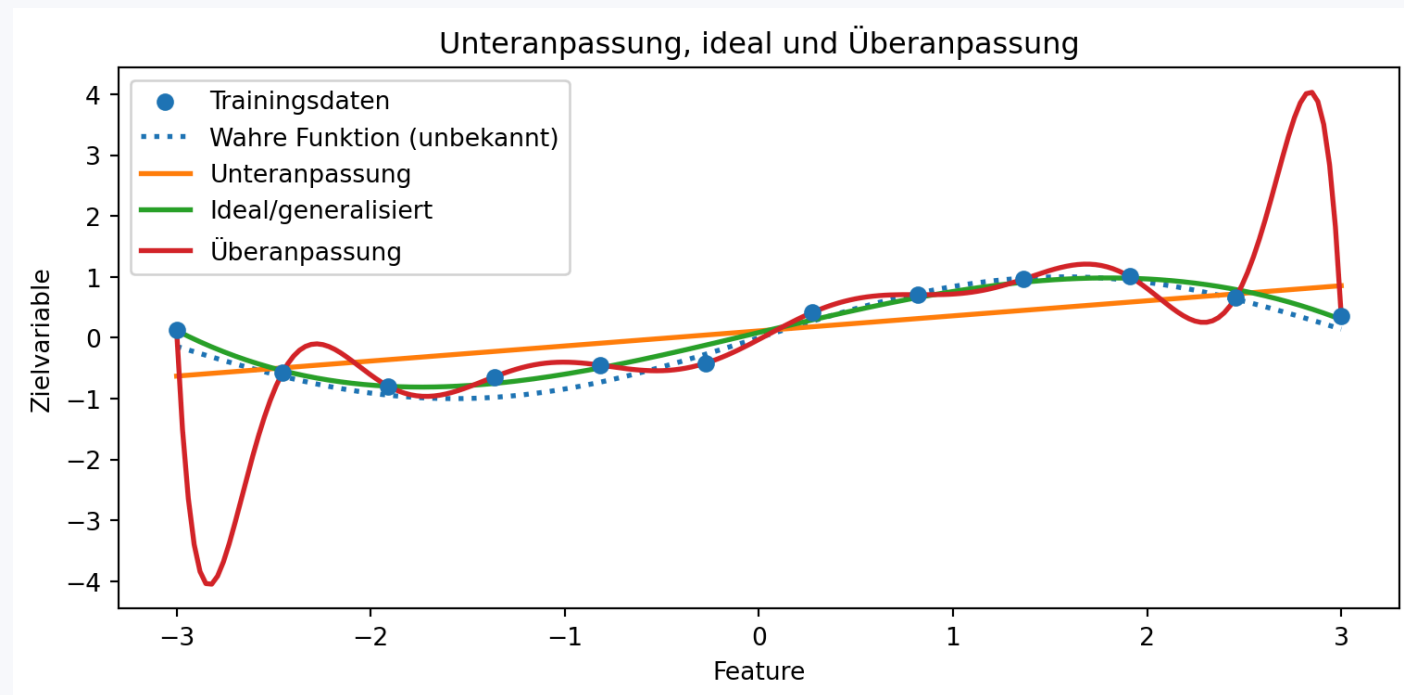


Maschinelles Lernen

- Maschinelles Lernen findet Regeln/Funktionen, die Eingabedaten auf gewünschte Ausgaben abbilden – ohne dass wir den exakten Algorithmus vorgeben.
- Wir trainieren ein Modell auf Beispielpaaren: (Eingabe, erwartete Ausgabe).
- **Überwachtes Lernen (Supervised Learning)** = Lernen aus gelabelten Daten.
- Die unbekannte „wahre“ Funktion wird durch ein Modell angenähert, das wir auf Daten fitten.

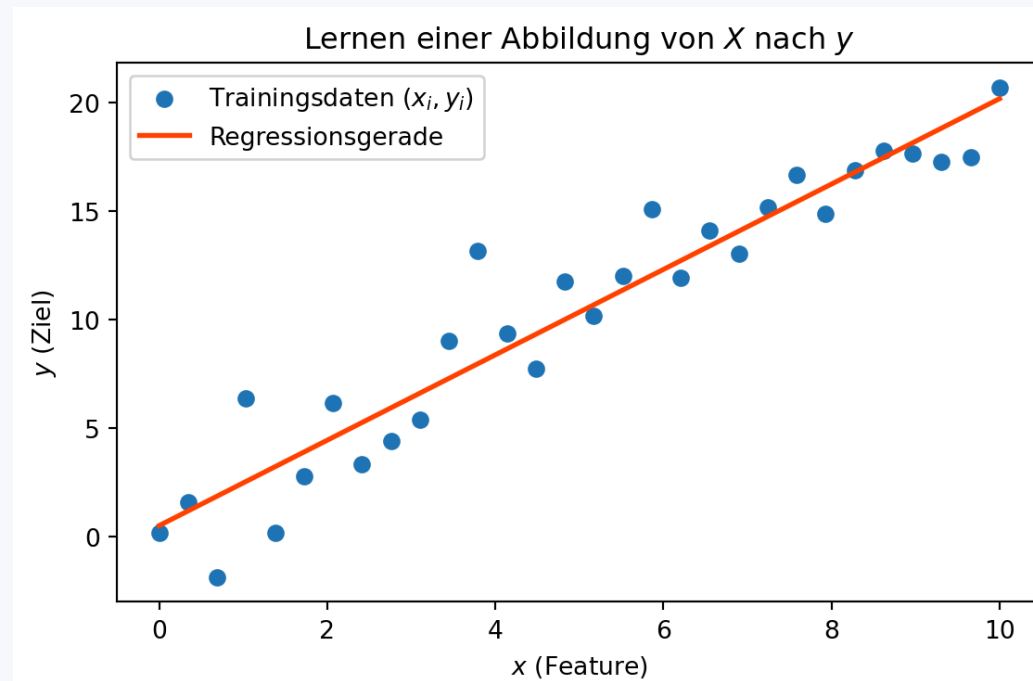
Overfitting und Underfitting

- **Underfitting:** Modell ist zu einfach – verpasst Muster.
- **Overfitting:** Modell ist zu komplex – merkt sich Trainingsdaten, versagt bei neuen Daten.
- **Generalisation:** Ziel ist ein niedriger Fehler auf neuen/ungesehenen Daten.
- Wir steuern die Modellkomplexität („Kapazität“) und nutzen Feature-Engineering sowie mehr Daten.



Maschinelles Lernen

- **Maschinelles Lernen** lernt eine Funktion, die Eingabefeatures X auf Ziel y abbildet.
- **Lineare Regression** modelliert $y = w^\top x + b + \epsilon$, wobei w Gewichte, b Bias und ϵ Rauschen sind.



Verlustminimierung (Loss Optimization)

- Ziel: Finde w, b , sodass das Modell die Trainingsdaten gut erklärt **und** gut generalisiert.
- **MLE/Verlust-Minimierung:** Häufig minimieren wir die mittlere quadratische Abweichung (MSE):

$$L(w, b) = \frac{1}{N} \sum_{i=1}^N \left(y_i - (w^\top x_i + b) \right)^2$$

Training per Gradientabstieg

- **Gradientabstieg** aktualisiert w, b iterativ in Richtung fallender Loss.
- **Mini-Batch-Gradientabstieg** nutzt kleine Batches als Schätzer des Gesamtgradienten – effizient und robust.

Training

- In jedem Schritt werden die Parameter wie folgt aktualisiert:

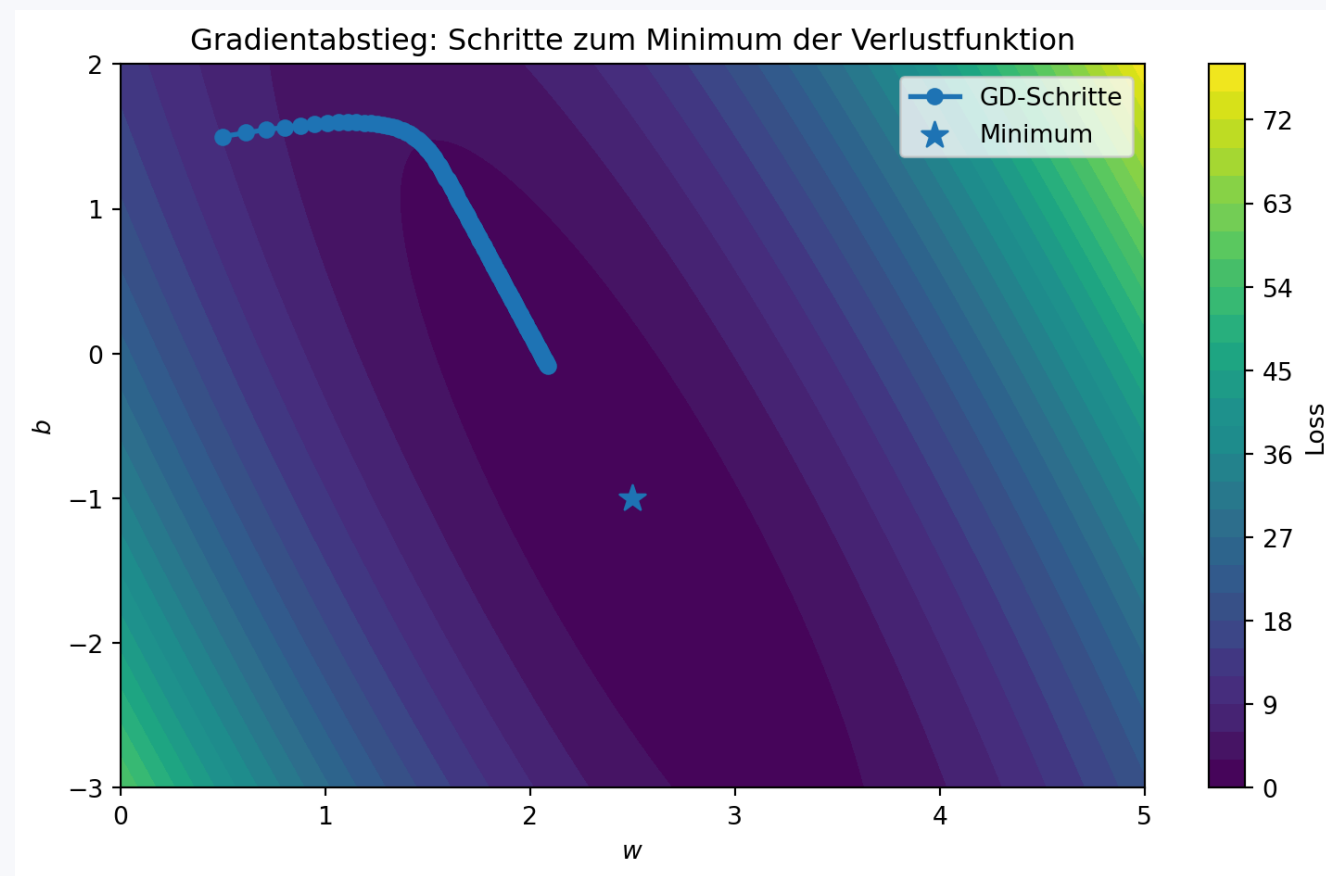
$$w \leftarrow w - \eta \frac{\partial L_{\text{batch}}}{\partial w}$$

$$b \leftarrow b - \eta \frac{\partial L_{\text{batch}}}{\partial b}$$

- η ist die Lernrate; L_{batch} ist der Verlust über einen zufällig gezogenen Mini-Batch.

- Diese Updates werden wiederholt, bis ein (lokales) Minimum der Loss erreicht ist.





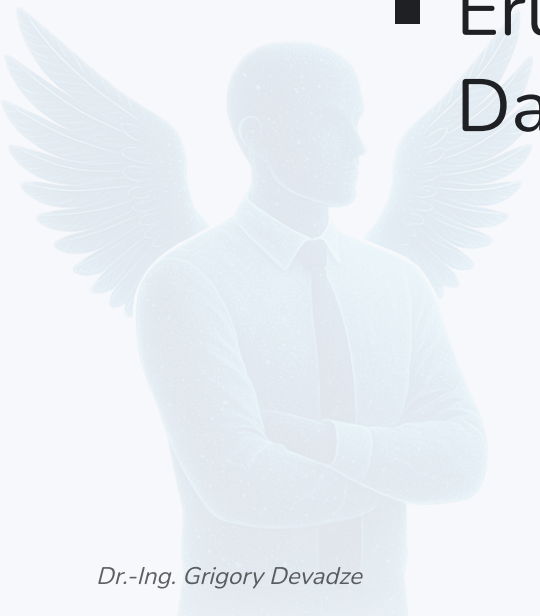
Von Linearer Regression zu Neuronalen Netzen

- **Lineare Regression:** lernt eine einfache Funktion

$$y = w^T x + b$$

- Erfasst nur lineare Zusammenhänge
- Begrenzte Flexibilität; komplexe, nichtlineare Muster bleiben unmodelliert

- **Neuronale Netze:** erweitern dies um Schichten („Neuronen“) und nichtlineare Aktivierungen
 - Können hochkomplexe Funktionen approximieren
 - Erlauben Lernen aus reichhaltigen, strukturierten Daten



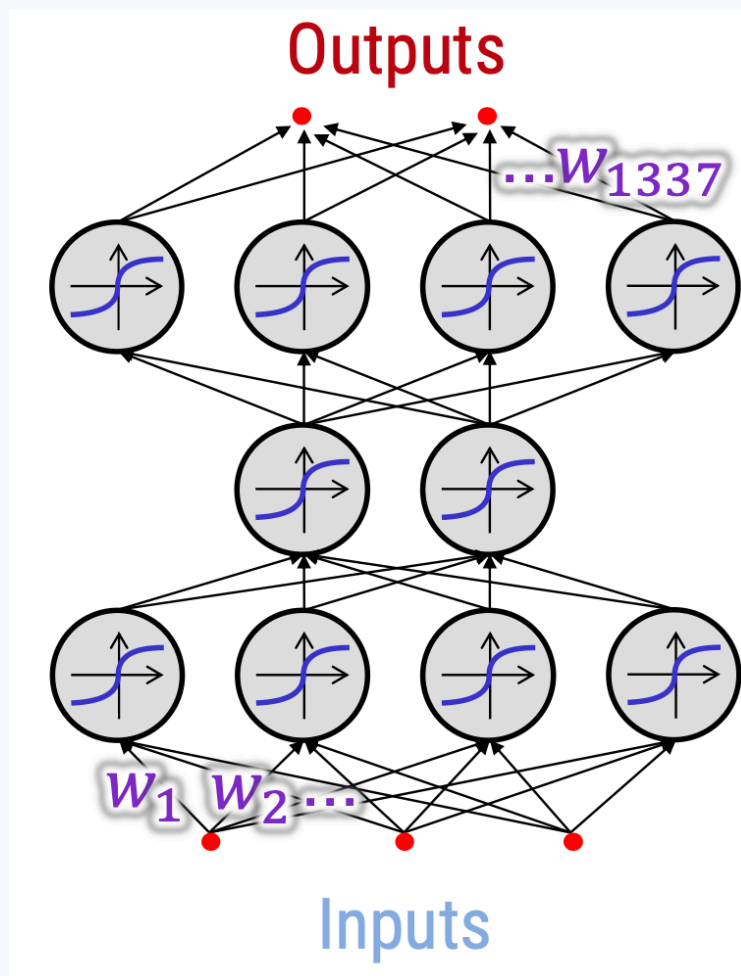
Neuronales Netz als verallgemeinerte Regression

- Ein neuronales Netz lässt sich als **Stack von Regressionen** mit nichtlinearen Transformationen je Schicht betrachten.

$$y = f^{(N)} \left(W^{(N)} f^{(N-1)} (\dots f^{(1)} (W^{(1)} x + b^{(1)}) \dots) + b^{(N)} \right)$$

- Dabei gilt:
 - x = Eingabefeatures
 - $W^{(\ell)}, b^{(\ell)}$ = Gewichte und Bias je Schicht ℓ
 - $f^{(\ell)}$ = Aktivierungsfunktion (z. B. ReLU, Sigmoid)
 - N = Anzahl der Schichten (Tiefe)



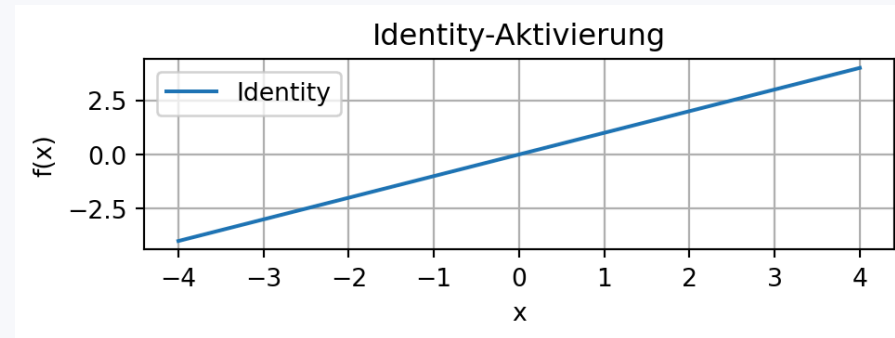


Aktivierungsfunktionen in Neuronalen Netzen

- Aktivierungsfunktionen bringen Nichtlinearität und Flexibilität.
- Häufige Varianten: **Identity**, **ReLU**, **Sigmoid**, **tanh**, **Softmax**
- Die passende Wahl hängt von Aufgabe und Schicht ab.

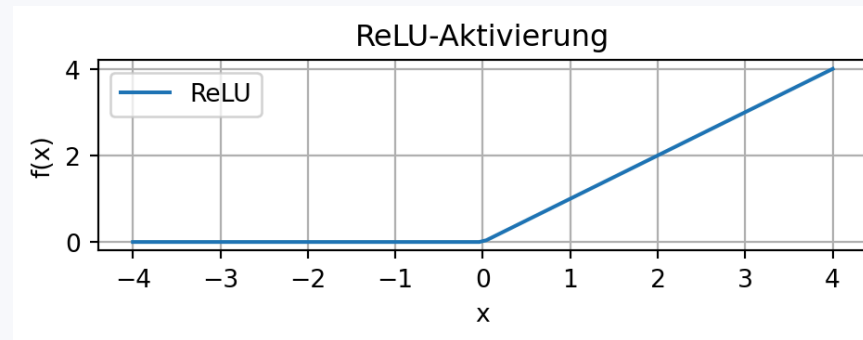
Identity (Identität)

- Formel: $f(x) = x$
- Linear, keine Nichtlinearität
- Z. B. Ausgabeschicht bei Regression (unbeschränkt)



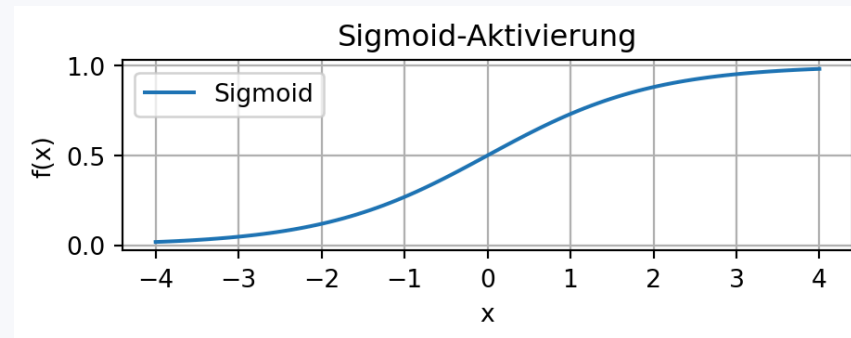
Rectified Linear Unit (ReLU)

- Formel: $f(x) = \max(0, x)$
- Beliebt in Hidden-Schichten tiefer Netze; effizient, reduziert Vanishing Gradients



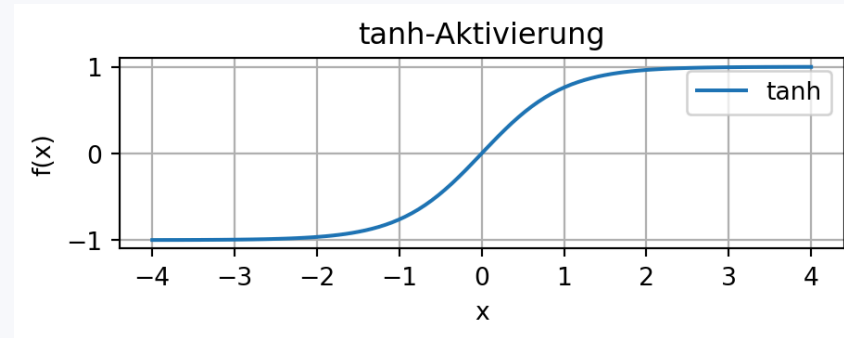
Sigmoid

- Formel: $f(x) = \frac{1}{1+e^{-x}}$
- Ausgabe zwischen 0 und 1 (Wahrscheinlichkeit)
- In der binären Ausgabeschicht gebräuchlich



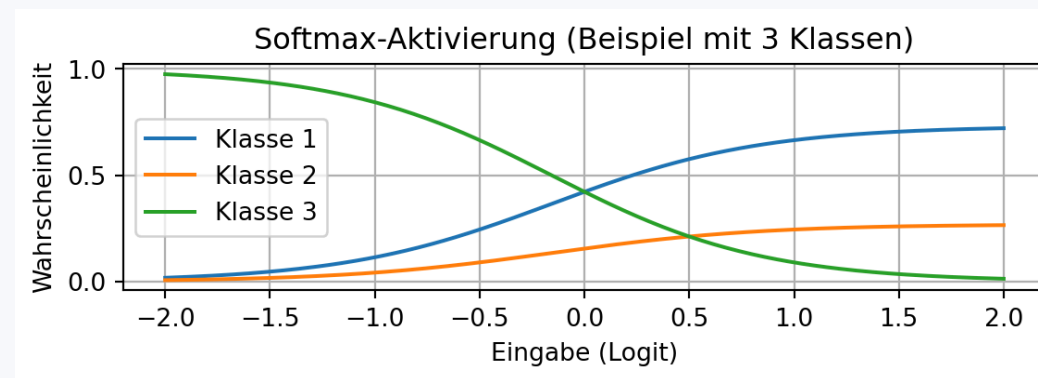
Hyperbolische Tangens (tanh)

- Formel: $f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- Ausgabe zwischen -1 und 1
- Teils in Hidden-Schichten und LSTM genutzt



Softmax

- Formel: $f_k(x) = \frac{e^{x_k}}{\sum_{j=1}^K e^{x_j}}$
- Wandelt einen K -dimensionalen Logit-Vektor in Wahrscheinlichkeiten um (Summe = 1)
- In der Ausgabeschicht für Multiklassen-Klassifikation



Was ist One-Hot-Encoding?

- **One-Hot-Encoding** repräsentiert Kategorien/Klassen als Vektoren:
 - Für K Klassen ist jede Klasse ein K -Vektor mit einer **1** und sonst **0**.
 - Beispiel (3 Klassen):
 - Klasse 1: $[1, 0, 0]$
 - Klasse 2: $[0, 1, 0]$
 - Klasse 3: $[0, 0, 1]$

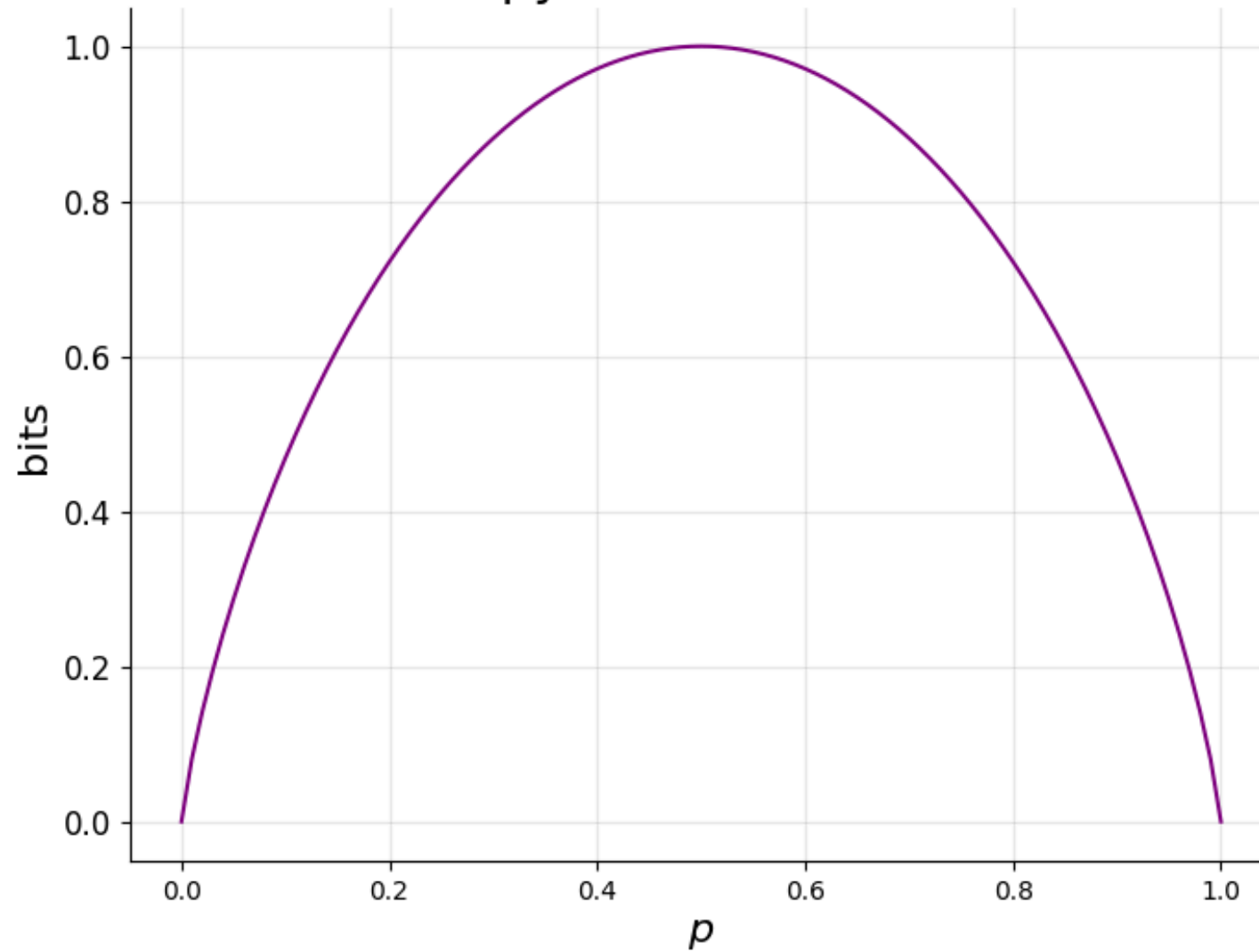
Binary Cross-Entropy (BCE)

- Für **binäre Klassifikation** (Labels 0/1)
- Vergleicht prognostizierte Wahrscheinlichkeit ($\hat{y}$) mit dem wahren Label (y):

$$\text{BCE}(y, \hat{y}) = - [y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

- Kleine Loss bei sicheren/korrekten Vorhersagen; hohe Loss bei falschen/sicheren Vorhersagen

Entropy of a Bernoulli trial



Informationsentropie

Categorical Cross-Entropy (CE)

- Für **Multiklassen-Klassifikation** (Label ist eine Klasse aus K)
- Mit One-Hot-Label y und Prognosen $\hat{y}$

$$\text{CE}(y, \hat{y}) = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

- Bestraft niedrige vorhergesagte Wahrscheinlichkeit für die wahre Klasse

Wann welche Loss?

- **BCE:** Binäre Ausgabe (0/1), Sigmoid in der letzten Schicht
- **CE:** Mehrere Klassen $(1, \dots, K)$, Softmax in der letzten Schicht
- **LLMs:** Standard ist Categorical Cross Entropy (CE) auf das nächste Token

Batch-Normalisierung: Trainingsbeschleunigung und Stabilität

- **BatchNorm** normalisiert die Aktivierungen für jedes Mini-Batch
- Zentriert die Eingaben jeder Schicht auf Mittelwert 0 und Varianz 1 und skaliert sie anschließend mit lernbaren Parametern

Wie funktioniert BatchNorm?

- Für jede Batch-Aktivierung x :

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \hat{x}_i + \beta$$

- γ, β werden gelernt; ϵ verhindert Division durch Null

Warum BatchNorm verwenden?

- Macht das Training schneller und robuster
- Ermöglicht höhere Lernraten und bessere Initialisierung

